

Titulación: GRADO INGENIERÍA  
INFORMÁTICA  
Año Académico: 2024/25 -- Curso: 3º  
Asignatura: Sistemas Distribuidos

uc3m

Universidad  
**Carlos III**  
de Madrid

|           |                        |      |           |
|-----------|------------------------|------|-----------|
| Alumno/a: | Álvaro García Piqueras | NIA: | 100495760 |
| Alumno/a: | Mario Collazo Naranjo  | NIA: | 100495949 |



## Memoria Práctica Final - Diseño e implementación de un sistema peer-to-peer

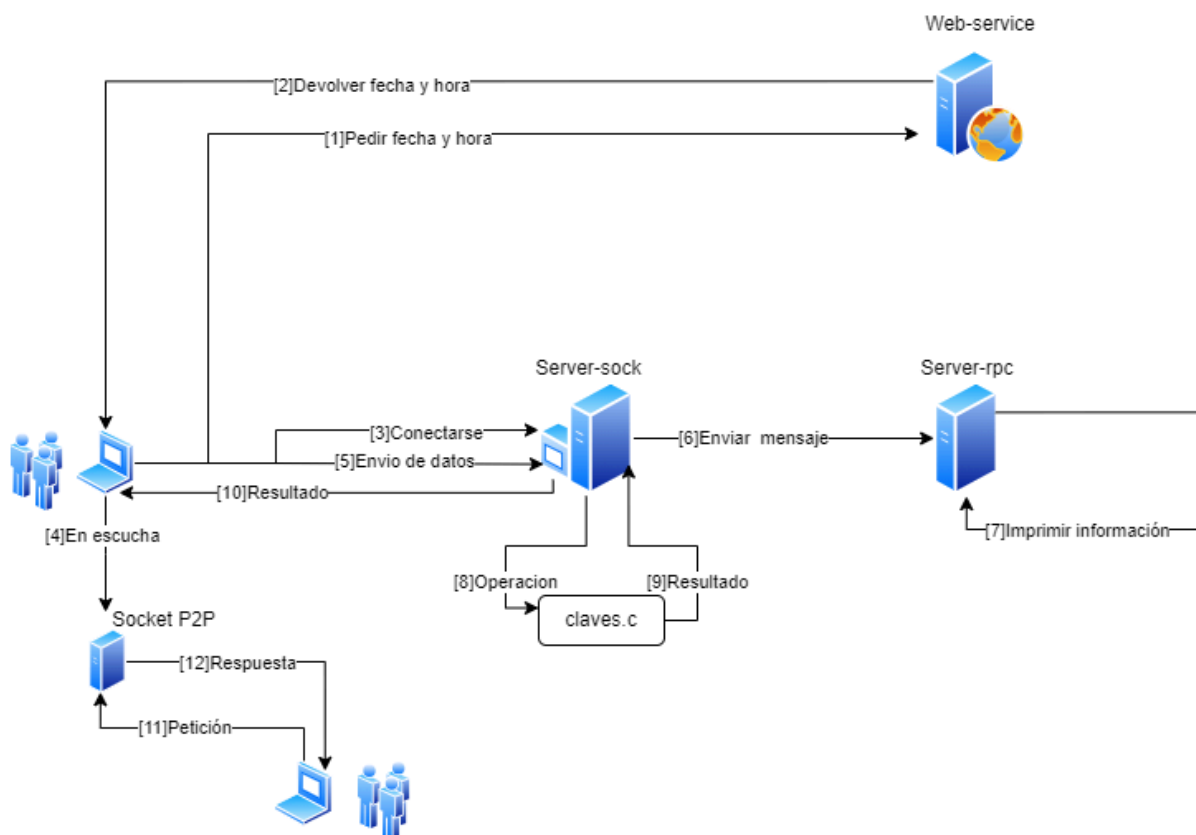
## ÍNDICE:

|                                                                               |           |
|-------------------------------------------------------------------------------|-----------|
| <b>1. Diagrama de la arquitectura completa</b>                                | <b>2</b>  |
| <b>2. Estructura del paso de mensaje</b>                                      | <b>5</b>  |
| <b>3. Justificación del paradigma seleccionado y protocolos de aplicación</b> | <b>7</b>  |
| <b>4. Operaciones de cada componente de la arquitectura</b>                   | <b>9</b>  |
| <b>5. Batería de pruebas</b>                                                  | <b>13</b> |
| <b>6. Consideraciones adicionales</b>                                         | <b>15</b> |
| <b>7. Conclusiones</b>                                                        | <b>16</b> |

## 1. Diagrama de la arquitectura completa

Este diagrama representa el flujo general de comunicación en un sistema distribuido. Los usuarios pueden solicitar la fecha y hora a un Web-service, conectarse a un servidor de sockets (Server-sock) para enviar datos, o comunicarse entre ellos mediante un sistema P2P. El Server-sock puede interactuar tanto con un módulo de procesamiento local (claves.c) como con un servidor RPC (Server-rpc), que se encarga de imprimir o procesar mensajes adicionales.

Es importante destacar que, aunque el diagrama utiliza numeración para identificar cada interacción, estos números no reflejan necesariamente el orden estricto de ejecución del programa. Debido al uso de hilos (threads), muchas de estas acciones pueden ejecutarse de forma concurrente, permitiendo que varios procesos ocurran simultáneamente o en orden variable según el contexto de ejecución.



1. El cliente solicita la fecha y hora a un **Web-service** externo.
2. El **Web-service** responde al cliente con la fecha y hora actual.
3. El cliente se conecta al servidor de sockets (**Server-sock**) para iniciar una operación.
4. El cliente entra en modo "escucha" para poder recibir peticiones de otros clientes a través del sistema **P2P**.
5. El cliente realiza un envío de datos al **Server-sock**.
6. El **Server-sock** reenvía el mensaje al **Server-rpc**, que actúa como log.
7. El **Server-rpc** imprime la información recibida.
8. El **Server-sock** invoca una operación sobre el módulo **claves.c**, que contiene la lógica de procesamiento.
9. El módulo **claves.c** devuelve el resultado de la operación al **Server-sock**.
10. El **Server-sock** envía el resultado final al cliente.
11. Otro cliente envía una petición **P2P** al cliente que está en modo escucha.
12. El cliente responde a esta petición con una respuesta **P2P**, completando la comunicación entre pares.

## 2. Estructura del paso de mensaje

En este sistema distribuido se utilizan varios tipos de mensajes entre los distintos componentes (cliente, servidor, servidor rpc, pares y servicio web). Los mensajes entre cliente-servidor se transmiten a través de sockets TCP, y la comunicación entre el cliente y el web-service mediante SOAP. Además, la comunicación servidor-servidor se hace mediante RPC.

### Comunicación del cliente:

La comunicación entre el cliente y el servidor se realiza mediante sockets TCP. El cliente envía cadenas que representan comandos (terminado en `\x00` ya que se manda desde python), después, se hace una llamada a la función conectada con el web service(se explica más adelante) y tras recibir una cadena con la fecha y la hora actual, se manda a través del socket. Luego manda el nombre de su usuario, y por último los campos necesarios dependiendo de la función que se esté ejecutando.

Las funciones que mandan cadenas distintas las siguientes:

- Register, unregister, connect, disconnect, list\_content: En estas funciones solo se pasa otro username como cadena de caracteres.
- Publish: A parte del usuario, se mandan cadenas de caracteres para el nombre del fichero y para la descripción.

Después de mandar los mensajes, el cliente espera a recibir un número entero que debe serializar. Cabe destacar que solo serializa el resultado obtenido por el servidor ya que es un int, y ocupa 4 bytes, por lo que cuando se pasa a la red, se debe cambiar al formato de la red porque se pueden desordenar. En cambio, cuando pasamos una cadena de caracteres, la pasamos char a char, y como solo ocupa un byte, no hay problemas ya que no se puede desordenar.

En cuanto a la comunicación entre el cliente y el web-service, se utiliza la librería Zeep que se encarga de la comunicación. El cliente hace una llamada y recibe un string en formato de fecha ISO 8601. El funcionamiento es bastante simple, el cliente se conecta a una url definida por nosotros, conectado a un puerto en local y el web service devuelve la fecha y hora actual todo con la librería.

Por último, queda la conexión cliente-cliente(peer-to-peer), la cual es casi igual que la conexión cliente-servidor, excepto porque cambia el puerto del socket. Cuando se ejecuta la función, el cliente manda a través del socket del otro cliente el comando de `get_file` y el nombre del fichero remoto. Después se pone a escuchar la

respuesta del otro cliente, serializa(ya que es de tipo int la primera respuesta) y luego se encarga de ir escribiendo todos los mensajes que le pasa el otro cliente. Asimismo, el cliente como está disponible para el resto de clientes, cuando le llega una petición de `get_file`, este comprueba que su fichero está disponible, y si lo está manda un "0" serializado de respuesta para indicar que va a proceder a mandarle todos los contenidos del fichero y después, le manda todo el contenido sin serializar.

### **Comunicación del servidor:**

El servidor por su parte solo tiene comunicaciones a través del socket tcp y a través de rpc con el servidor rpc.

Como ya sabemos, el servidor primero recibe una llamada a una función a través del socket, y luego recibe el resto de argumentos necesarios para ejecutar la función. Como todo lo que le tiene que llegar son cadenas de caracteres no es necesario serializar los datos. Cuando le llegan todos los argumentos, este llama a la función correspondiente en `claves.h` y devuelve, primero, el resultado de la operación (serializado ya que es de tipo int) y luego el resto de parámetros necesarios en forma de cadena.

En cuanto a la parte relacionada con RPC, el servidor procede a pasarle la fecha y hora al servidor RPC mediante las funciones definidas en `log_clnt`, y a través de la propia serialización que ofrece RPC el mensaje llega adecuadamente y preparado para ser tratado por el servidor RPC, el cual lo imprime por pantalla y no devuelve nada al servidor.

### 3. Justificación del paradigma seleccionado y protocolos de aplicación

Nuestra arquitectura distribuida está basada en múltiples paradigmas de comunicación. Se ha combinado el uso de sockets TCP, comunicación P2P, invocación remota mediante ONC-RPC y una consulta a un servicio web HTTP. Esta combinación permite cubrir distintos tipos de interacción entre componentes del sistema, desde la comunicación directa entre pares hasta la interacción con servicios externos. Vamos a tratar cada método por separado:

- **Cliente-Servidor (Sockets TCP):** Los clientes, implementados en Python, se comunican con el servidor principal desarrollado en C mediante sockets TCP. Este protocolo orientado a la conexión proporciona una comunicación fiable, ordenada y con control de errores. Se utiliza para registrar, conectar, publicar o consultar datos.
- **P2P (Sockets TCP entre clientes):** Para habilitar el intercambio de archivos y la colaboración descentralizada, se ha implementado un mecanismo P2P utilizando también sockets TCP, pero directamente entre clientes. Esto es útil, por ejemplo, para el intercambio de datos o archivos de manera descentralizada, reduciendo la carga sobre el servidor central.
- **RPC (ONC-RPC en C):** El servidor central realiza llamadas remotas a un segundo servidor especializado mediante ONC-RPC (Open Network Computing Remote Procedure Call). Este paradigma de invocación remota encapsula funciones específicas, como el registro de eventos, en un módulo separado que puede ser invocado a través de la red como si fueran funciones locales.
- **Webservice (HTTP):** Para obtener la fecha y hora exacta en cada operación, se realiza una llamada a un servicio web externo. Esto introduce el uso del protocolo HTTP como parte del sistema.

La combinación de estos paradigmas y protocolos responde a necesidades concretas del sistema, con Sockets TCP se asegura una comunicación directa, eficiente y confiable. Con P2P se potencia la descentralización y el rendimiento. Con ONC-RPC permite la escalabilidad de servicios. HTTP proporciona interoperabilidad con servicios web actuales.

## 4. Operaciones de cada componente de la arquitectura

**Client.py:** Este archivo implementa la lógica completa de un cliente dentro de una red de compartición de archivos distribuida (P2P), en la que los clientes pueden comunicarse tanto con un servidor central como directamente con otros clientes pares. Para lograr esto, el primer paso es establecer una conexión con el servidor mediante el método `connect_server()`, que crea un socket TCP/IP y se conecta a una dirección IP y puerto específicos.

Una vez establecida la conexión, se utilizan métodos como `send_data()` para enviar mensajes al servidor, `receive_data()` para recibir respuestas, y `receive_until_null()` para seguir recibiendo datos hasta que se detecte un carácter nulo (`\0`), lo que indica el final del mensaje. Esto es importante porque en las comunicaciones por red, los datos pueden llegar en fragmentos, y esta función garantiza que se recibe el mensaje completo.

Para llevar un registro temporal de las acciones del cliente, se incluye el método `get_datetime()`, que utiliza la biblioteca `zeep` para hacer una solicitud a un servicio web SOAP, obteniendo así la hora y fecha actual estandarizadas.

Las funciones principales incluyen la gestión de usuarios y contenidos. Las funciones `register_user()` y `unregister_user()` permiten registrar o eliminar un usuario en el sistema enviando las órdenes al servidor central. Con `connect()` y `disconnect()` el cliente inicia o cierra sesión, y el servidor actualiza el estado de conexión del usuario. Al subir un archivo, `publish()` envía los metadatos del archivo (nombre, ruta, descripciones) al servidor, `delete()` hace lo opuesto, elimina un archivo del catálogo. `list_users()` y `list_content()` permiten consultar qué usuarios están conectados y qué archivos están disponibles para descarga en ese momento.

Finalmente, `get_file()` permite al cliente descargar un archivo directamente desde otro cliente (no desde el servidor), siempre y cuando el servidor confirme que ese archivo está publicado y el usuario que lo comparte está conectado. Para permitir esta transferencia directa de archivos entre clientes, se implementa un pequeño servidor en cada cliente. La función `start_server_socket()` crea un socket servidor local que escucha conexiones entrantes en un puerto aleatorio, y lanza un hilo que atiende múltiples peticiones concurrentes de otros clientes mediante hilos (`threading`).



Cuando un cliente se conecta para solicitar un archivo, el hilo invoca `handle_peer_connection()`, que gestiona la petición del archivo. Esta función primero verifica que el archivo solicitado existe en el sistema local; si es así, obtiene su tamaño y lo envía por bloques binarios (chunks) de 1024 bytes usando un bucle de lectura con `open(file_path, 'rb')`. Así, se logra una comunicación eficiente y descentralizada entre clientes, transfiriendo archivos directamente sin pasar por el servidor central.

**Datetime\_service.py:** Este archivo define un servicio web SOAP usando la biblioteca Spyne, que proporciona la fecha y hora actual. La clase `DateTimeService` expone el método `get_datetime()`, que devuelve la fecha y hora formateadas usando `datetime.now()`. El servicio se publica como una aplicación WSGI, utilizando el protocolo SOAP 1.1. Finalmente, se lanza un servidor local en el puerto 8000 que atiende las peticiones SOAP.

**Server-sock.c:** Este archivo implementa el servidor central para el sistema distribuido. Está diseñado para gestionar las conexiones de múltiples clientes que interactúan con el servidor a través de un protocolo de red basado en sockets TCP.

El servidor escucha en un puerto especificado por el usuario y, cada vez que un cliente se conecta, crea un nuevo hilo que se encarga de procesar la solicitud del cliente de manera concurrente, permitiendo que el servidor maneje varias conexiones al mismo tiempo sin bloquearse.

Cuando un cliente se conecta, el servidor lo acepta y asigna un hilo a esa conexión para procesar las operaciones solicitadas. El programa utiliza hilos para manejar de manera eficiente múltiples clientes simultáneamente.

Cada hilo ejecuta la función `tratar_mensaje`, que se encarga de recibir datos del cliente, interpretarlos y ejecutar las operaciones correspondientes.

Cada operación también genera una llamada RPC (Remote Procedure Call) al servidor de registro, utilizando la función `enviar_rpc`. Esto asegura que las operaciones realizadas por los clientes se registren adecuadamente en un sistema de log remoto, permitiendo llevar un control de las actividades que se realizan en el sistema.

Finalmente, al recibir una solicitud, el servidor responde con los resultados de la operación, ya sea con éxito o error, y cierra la conexión una vez que la operación ha sido procesada.

**log.x:** Este archivo define la interfaz RPC, define una estructura mensaje que contiene cuatro campos de tipo string con un máximo de 256 caracteres: usuario, operacion, publicacion y fecha. Estos campos representan los datos de una operación realizada por un usuario. El programa OPERACIONLOG define una versión VERSION\_MENSAJE, que incluye una función remota llamada REGISTRAR\_OP, identificada con el número 1, la cual recibe un objeto de tipo mensaje y devuelve un entero como resultado (para indicar éxito o error). Esta interfaz permite registrar operaciones de usuarios en un sistema distribuido.

**log.h:** Este archivo se encarga de declarar la interfaz RPC para registrar operaciones de clientes. Define la estructura **mensaje**, asocia el programa RPC OPERACIONLOG con la versión VERSION\_MENSAJE, que incluye el procedimiento remoto REGISTRAR\_OP. También declara funciones cliente-servidor y la función xdr\_mensaje para serializar datos.

**log\_xdr.c:** Este archivo define la función xdr\_mensaje, que se encarga de la serialización y deserialización de la estructura mensaje para su transmisión por RPC.

**log\_svc.c:** Este archivo define la función operacionlog\_1, que despacha las peticiones entrantes, deserializa los datos, llama a la función correspondiente (registrar\_op\_1\_svc) y envía la respuesta. En main, se configuran los servicios TCP y UDP, se registran con el sistema de RPC y se inicia el bucle de espera con svc\_run para atender solicitudes remotas.

**log\_clnt.c:** Este archivo define el stub del cliente RPC para la función remota registrar\_op\_1. Utiliza clnt\_call para enviar una solicitud al servidor, pasando un objeto mensaje y recibiendo un entero como respuesta.

**servidor-rpc.c:** Este archivo implementa la función del servidor registrar\_op\_1\_svc, que es invocada cuando se recibe una llamada RPC desde el cliente. La función recibe un mensaje y lo imprime en la consola. Devuelve true indicando que la operación fue procesada correctamente.

**lines.h:** Este archivo declara las funciones utilizadas en la comunicación entre procesos o redes mediante sockets.

**lines.c:** Este archivo implementa dos funciones para la comunicación de mensajes mediante sockets. `sendMessage()` envía un mensaje a través de un socket. Usa un bucle para asegurarse de que se envíen todos los bytes, devolviendo 0 si tiene éxito y -1 en caso de error. `recv until null()` Lee caracteres del socket hasta encontrar un carácter nulo ('\0') o alcanzar el tamaño máximo. Devuelve el número de bytes leídos o -1 si ocurre un error.

**claves.h:** Este archivo define estructuras y funciones para gestionar usuarios y ficheros. La estructura `Fichero` almacena el nombre de un archivo y su siguiente en la lista, mientras que `Usuario` guarda información como nombre, IP, puerto, estado de conexión y los ficheros publicados.

**claves.c:** Este archivo implementa un sistema para gestionar usuarios y archivos en un entorno multihilo, donde cada usuario tiene un nombre, una IP, un puerto y un estado de conexión, junto con una lista de archivos que ha publicado. Utiliza un mutex para garantizar que las operaciones sobre los usuarios y sus archivos se realicen de manera segura en un entorno concurrente. Las funciones permiten registrar, eliminar, conectar y desconectar usuarios, así como publicar y eliminar archivos asociados a ellos. Además, ofrece la capacidad de listar los usuarios conectados y los archivos publicados por un usuario específico, asegurando que solo los usuarios conectados puedan interactuar con estos recursos.

## 5. Batería de pruebas

Para ejecutar las pruebas debemos tener:

- En la terminal del server se debe crear la variable de entorno mediante: **export LOG\_RPC\_IP=localhost**
  - Una terminal donde ejecutaremos el Webservice mediante: **datetime\_service.py**
  - Una terminal donde ejecutaremos **MAKE** para la compilación del código y posteriormente **./server <port>**
  - Una terminal donde después del **MAKE** ejecutaremos **./servidor-rpc**
  - Una terminal donde ejecutaremos los test mediante **python test\_n\_client.py -s <ip> -p <port>** siendo n el número del test deseado el port debe ser el mismo que el del servidor
- 
- **test\_1\_client.py:** En este test, se valida la funcionalidad básica de todas las operaciones cliente-servidor. Se ejecutan en orden lógico, simulando escenarios reales y errores típicos que podrían producirse en el uso de un sistema P2P.  
Se valida que un usuario se puede registrar correctamente, que no se permite registrar el mismo nombre dos veces, y que se puede eliminar (desregistrar) del sistema. También se comprueba que el desregistro de un usuario inexistente no genera errores graves.  
Se prueba la conexión de usuarios registrados, así como la gestión de errores al intentar conectar usuarios inexistentes o ya conectados. Del mismo modo, se valida la desconexión en distintos estados.  
Se testean las condiciones normales y erróneas de publicación de archivos: publicar uno nuevo, intentar publicar el mismo archivo dos veces, publicar sin estar conectado o registrado, etc. Luego se comprueba el borrado correcto del archivo y qué ocurre al borrar uno no publicado o desde estados inválidos.  
Se evalúa si se puede pedir correctamente la lista de usuarios conectados, y qué ocurre si lo solicita un usuario no registrado o desconectado. También se comprueba la capacidad de listar los archivos publicados por un usuario concreto (propio o remoto), en distintos contextos.
  - **test\_2\_client.py:** En este test, se valida la funcionalidad del mecanismo P" P(peer-to-peer) implementado en el cliente. Esta prueba simula el funcionamiento del sistema de intercambio de archivos entre usuarios registrados y conectados, utilizando sockets TCP directamente entre clientes (peers).

1. Se inicia el socket de escucha, lo cual permite a este cliente actuar como servidor peer, capaz de recibir conexiones de otros clientes que desean descargar archivos.
  2. Este usuario se registra, se conecta y publica un archivo llamado "archivo\_prueba.txt". Este es el archivo que se ofrecerá a otros clientes.
  3. Se registra y conecta un segundo usuario, que solicita la lista de usuarios conectados. Consulta los archivos publicados por "user1". Intenta descargar el archivo "archivo\_prueba.txt" usando getfile.
  4. Finalmente, se lanza una prueba negativa solicitando un archivo que no existe ("archivo\_no\_existe.txt"), con el fin de validar que el sistema responde correctamente a errores esperados, como archivos inexistentes.
- **test\_3\_client.py:** En este test, se valida la concurrencia del programa, está diseñado para evaluar cómo el servidor maneja múltiples peticiones simultáneas que compiten por registrar el mismo nombre de usuario. El propósito es verificar que el sistema es seguro frente a condiciones de carrera (race conditions) cuando varios clientes intentan registrar el mismo nombre de usuario al mismo tiempo.  
Para ello se crean 10 hilos cada uno intentando registrar a un mismo usuario. Todos los hilos son lanzados simultáneamente, el servidor debe únicamente aceptar uno de ellos y rechazar el resto, validando así que el sistema es robusto frente a accesos concurrentes sin corrupción del estado.

## 6. Consideraciones adicionales

Como consideraciones adicionales, podemos hablar de alguna decisión de diseño que hemos tomado en el transcurso del desarrollo de la práctica:

- Desconexión cuando haces QUIT: En el enunciado de la práctica, no se especifica que cuando se hace quit, el usuario se desconecta automáticamente, pero es un detalle que hemos considerado obvio y lo resolvimos haciendo una llamada a disconnect, ya que así lo hemos considerado.
- Durante la implementación del servicio web, utilizamos el puerto 8000 como punto de escucha del servidor HTTP. Sin embargo, este tipo de puertos son comúnmente utilizados en entornos de desarrollo y pueden estar ocupados por otros procesos, lo que puede provocar errores como OSError: [Errno 98] Address already in use.  
Por esta razón, tuvimos que cambiar a otro puerto libre, como 8080, que es también muy habitual en aplicaciones web de prueba.
- Otra consideración a destacar es que hemos ejecutado la opción de rpcgen -M,
- Otra consideración relevante es el uso de la opción -M al ejecutar rpcgen. Esta opción genera un esqueleto de servidor RPC que evita la implementación automática de la función de liberación de resultados (\*\_freeresult), delegando dicha responsabilidad al programador. Como consecuencia, ha sido necesario modificar manualmente dicha función en el archivo svc.c para gestionar correctamente la liberación de memoria. Esto implica que no es posible eliminar y regenerar el archivo svc.c mediante rpcgen -M sin perder las modificaciones realizadas, a menos que se vuelva a editar manualmente tras cada generación.

## 7. Conclusiones

En esta práctica, hemos aprendido a integrar distintos sistemas distribuidos. Esta experiencia nos ha permitido comprender mejor cómo interactúan distintos componentes en una arquitectura compleja, utilizando sockets, comunicación P2P, llamadas remotas (RPC) y servicios web.

Como ya hemos visto a lo largo de este trabajo, hemos empleado varias arquitecturas enfocadas en el paso de mensajes entre programas. Esto es muy interesante ya que hemos aprendido que se pueden coordinar varias arquitecturas simultáneamente y que funcione todo bien.

Durante el desarrollo, hemos trabajado con sockets TCP para establecer la comunicación entre el cliente en Python y el servidor en C, así como para permitir la comunicación directa entre clientes en un esquema P2P. También hemos utilizado llamadas a procedimientos remotos (RPC), y hemos accedido a un servicio web externo para obtener la fecha y hora actual. Esta combinación de tecnologías nos ha mostrado las ventajas de cada enfoque.

No obstante, también hemos podido apreciar algunas de sus limitaciones: la complejidad de sincronizar correctamente todos los componentes, ya que al tratarse de un sistema con varios módulos, la necesidad de seguir de forma rigurosa los protocolos de comunicación establecidos para evitar fallos en la integración ha resultado algo complicado de entender, pero relativamente fácil de implementar. A pesar de ello, el resultado nos ha permitido adquirir una visión mucho más completa y práctica sobre el diseño, implementación y coordinación de sistemas distribuidos reales.